



E4 Educator v2.0

T04

I2C fundamentals

Tier 1 · Tutorial 4 of 7

Walark Electronics · Fundrum Engineering · May 2026

In this tutorial

You will learn:

- How the I2C bus works — SDA, SCL, addressing and pull-ups
- Why I2C is the ideal entry point for sensor communication
- How to run an I2C scanner to find devices on the bus
- How to read temperature and humidity from the AHT20 sensor on the E4
- How to read barometric pressure from a BMP280 via the expansion header
- How to handle multiple I2C devices on the same bus without conflict
- How to build a timed sensor reading loop with `millis()` and Serial reporting

You will need:

- E4 Educator v2.0 board with ESP32 and AHT20 fitted
- T01, T02, T03 completed — `Button.h` available
- USB-C cable (data-capable)
- Optional: BMP280 module for the expansion header exercise

1 How I2C works

I2C (Inter-Integrated Circuit, pronounced "eye-squared-see") is a two-wire serial communication protocol designed for connecting sensors, displays, and peripherals to a microcontroller over short distances. It is the most widely used sensor interface in embedded systems — and the easiest to get started with.

1.1 The two wires

Signal	E4 GPIO	Purpose
SDA	GPIO 21	Serial Data — carries the actual data bytes bidirectionally between master and device
SCL	GPIO 22	Serial Clock — the ESP32 generates this to synchronise every bit transfer

That is the entire physical interface. Two wires, shared by every device on the bus simultaneously. This is I2C's great strength — you can connect many devices using the same two GPIO pins, with no additional wiring per device.

1.2 Addressing

With multiple devices sharing the same two wires, I2C needs a way to talk to one device at a time without the others responding. This is handled by addressing. Every I2C device has a 7-bit address — a unique number between 0x00 and 0x7F — burned into it by the manufacturer.

When the ESP32 wants to communicate with a device, it broadcasts the target address on the bus first. Only the device with that address responds. All others ignore the message.

Device	Default address	Notes
AHT20 (onboard)	0x38	Fixed address — cannot be changed
SSD1306 OLED	0x3C	Some modules offer 0x3D via solder bridge
BMP280	0x76 or 0x77	Depends on SDO pin — LOW = 0x76, HIGH = 0x77
BME280	0x76 or 0x77	Same as BMP280
BH1750 light sensor	0x23 or 0x5C	ADDR pin LOW = 0x23, HIGH = 0x5C

★ Key point

I2C addresses are fixed by the hardware. You cannot have two devices with the same address on the same bus — they will try to talk simultaneously and corrupt each other's data. Always check addresses before connecting multiple sensors. The I2C scanner in Section 2 reveals exactly what is on your bus.

1.3 Pull-up resistors

I2C uses open-drain signalling — devices can pull the bus LOW but cannot drive it HIGH. The bus needs pull-up resistors to 3.3V to return the lines to HIGH when no device is pulling them down.

On the E4, the AHT20 circuit and the I2C expansion headers include 4.7kΩ pull-up resistors on SDA and SCL. These are already fitted — you do not need to add external pull-ups for devices connected to the E4 I2C bus.

If you add many I2C devices

Each additional device adds parallel resistance to the bus pull-ups, lowering the effective pull-up value and potentially slowing signal rise times. For the E4 course this is not a concern — the number of devices is small and the bus is short. It is worth knowing for future designs.

2 The I2C scanner — always run this first

Before writing any sensor code, run the I2C scanner. It sweeps every possible address on the bus and reports which devices respond. This is the single most useful diagnostic tool for I2C work.

Create a new project called E4_I2C_Scanner and paste this into main.cpp:

```
#include <Arduino.h>
#include <Wire.h>

void setup() {
  Serial.begin(115200);
  Wire.begin(I2C_SDA, I2C_SCL);
  Serial.println("\nE4 I2C Scanner");
  Serial.println("Scanning...");

  int deviceCount = 0;

  for (byte addr = 1; addr < 127; addr++) {
    Wire.beginTransmission(addr);
    byte error = Wire.endTransmission();

    if (error == 0) {
      Serial.printf("  Device found at 0x%02X", addr);

      // Identify known devices
      if (addr == 0x38) Serial.print("  <- AHT20 temperature/humidity");
      if (addr == 0x3C) Serial.print("  <- SSD1306 OLED display");
      if (addr == 0x3D) Serial.print("  <- SSD1306 OLED (alt address)");
      if (addr == 0x76) Serial.print("  <- BMP280/BME280 pressure sensor");
      if (addr == 0x77) Serial.print("  <- BMP280/BME280 (alt address)");
      if (addr == 0x23) Serial.print("  <- BH1750 light sensor");

      Serial.println();
      deviceCount++;
    }
    else if (error == 4) {
      Serial.printf("  Unknown error at 0x%02X\n", addr);
    }
  }

  if (deviceCount == 0) {
    Serial.println("No I2C devices found.");
    Serial.println("Check wiring and pull-up resistors.");
  } else {
    Serial.printf("\nScan complete. %d device(s) found.\n", deviceCount);
  }
}

void loop() {
  // Scanner runs once in setup()
  // Press RST to rescan
}
```

With only the AHT20 fitted, you should see:

```
E4 I2C Scanner
Scanning...
  Device found at 0x38  <- AHT20 temperature/humidity

Scan complete. 1 device(s) found.
```

Nothing found?

Check that `Wire.begin()` uses the `I2C_SDA` and `I2C_SCL` defines from your `build_flags` — not the default Arduino pins. The E4 uses GPIO 21 (SDA) and GPIO 22 (SCL). If you see no devices, verify the `platformio.ini` `build_flags` contain `-DI2C_SDA=21 -DI2C_SCL=22`.

3 Reading the AHT20 — temperature and humidity

The AHT20 is a combined temperature and humidity sensor made by ASAIR. It is accurate, reliable, and simple to use. On the E4 it is permanently connected to the I2C bus — no wiring required.

3.1 Install the library

Add the AHT20 library to your platformio.ini lib_deps section:

```
[env:e4_educator_v2]
; ... existing settings ...
lib_deps =
    adafruit/Adafruit AHTX0
    adafruit/Adafruit BusIO
```

PlatformIO will download these automatically on the next build.

3.2 Basic AHT20 reading

```
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_AHTX0.h>
#include "Button.h"

Adafruit_AHTX0 aht;
Button nextBtn(BTN_NEXT);

// Timing
unsigned long lastRead = 0;
const unsigned long READ_INTERVAL = 2000; // Read every 2 seconds

void setup() {
    Serial.begin(115200);
    Serial.printf("T04 - AHT20 Reader FW: %s\n", FW_VERSION);

    Wire.begin(I2C_SDA, I2C_SCL);

    // Initialise LEDs
    pinMode(LED_GREEN, OUTPUT);
    pinMode(LED_RED, OUTPUT);
    digitalWrite(LED_GREEN, LOW);
    digitalWrite(LED_RED, LOW);

    nextBtn.begin();

    // Initialise AHT20
    if (!aht.begin(&Wire)) {
        Serial.println("ERROR: AHT20 not found. Check I2C bus.");
        digitalWrite(LED_RED, HIGH);
        while (true) { delay(100); } // Halt - sensor required
    }
```

```

digitalWrite(LED_GREEN, HIGH);
Serial.println("AHT20 ready. Reading every 2 seconds.");
Serial.println("Press NEXT to force an immediate reading.");
}

void loop() {
  unsigned long now = millis();
  Button::Event nextEv = nextBtn.read();

  // Read on timer or on button press
  bool shouldRead = (now - lastRead >= READ_INTERVAL);
  if (nextEv == Button::SHORT_PRESS) shouldRead = true;

  if (shouldRead) {
    lastRead = now;

    sensors_event_t humidity, temp;
    aht.getEvent(&humidity, &temp);

    Serial.printf("[%lu ms] Temp: %.1f°C Humidity: %.1f%%\n",
                  now, temp.temperature, humidity.relative_humidity);
  }
}

```

You should now see temperature and humidity readings every 2 seconds in the Serial monitor, with an immediate reading triggered by pressing NEXT.

3.3 AHT20 characteristics

Parameter	Detail
Temperature range	-40°C to +85°C, $\pm 0.3^\circ\text{C}$ accuracy
Humidity range	0–99.9% RH, $\pm 2\%$ accuracy
I2C address	0x38 — fixed, cannot be changed
Supply voltage	2.0–5.5V. E4 runs at 3.3V.
Measurement time	~75ms per measurement. The library handles this automatically.
Self-heating	Avoid reading faster than every 2 seconds — the sensor generates a small amount of heat during measurement that can affect readings at high speed.

4 Multiple devices on the I2C bus

One of I2C's greatest strengths is that multiple devices share the same two wires simultaneously. The E4 expansion headers give you easy access to the I2C bus to add additional sensors. Let's add a BMP280 pressure sensor alongside the AHT20.

4.1 Connecting a BMP280 to the expansion header

Connect the BMP280 module to the E4's AHT20-BMP280 expansion header:

BMP280 pin	E4 header pin	Notes
VCC	3v3	3.3V only — do not connect to 5V
GND	GND	Common ground
SDA	SDA	Shared I2C data line
SCL	SCL	Shared I2C clock line

Run the I2C scanner again after connecting. You should now see both devices:

```
E4 I2C Scanner
Scanning...
  Device found at 0x38  <- AHT20 temperature/humidity
  Device found at 0x76  <- BMP280/BME280 pressure sensor

Scan complete. 2 device(s) found.
```

4.2 Reading both sensors

Add the BMP280 library to platformio.ini:

```
lib_deps =
  adafruit/Adafruit AHTX0
  adafruit/Adafruit BMP280 Library
  adafruit/Adafruit BusIO
```

```
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_AHTX0.h>
#include <Adafruit_BMP280.h>
#include "Button.h"

Adafruit_AHTX0 aht;
Adafruit_BMP280 bmp(&Wire);
Button nextBtn(BTN_NEXT);

unsigned long lastRead = 0;
const unsigned long READ_INTERVAL = 3000;

void setup() {
  Serial.begin(115200);
```



```

Serial.printf("T04 - Dual sensor  FW: %s\n", FW_VERSION);

Wire.begin(I2C_SDA, I2C_SCL);

pinMode(LED_GREEN, OUTPUT);
pinMode(LED_RED,   OUTPUT);
bool ok = true;

if (!aht.begin(&Wire)) {
    Serial.println("ERROR: AHT20 not found");
    ok = false;
}

if (!bmp.begin(0x76)) {
    Serial.println("ERROR: BMP280 not found - check header connection");
    ok = false;
} else {
    // Recommended settings for weather monitoring
    bmp.setSampling(Adafruit_BMP280::MODE_NORMAL,
                    Adafruit_BMP280::SAMPLING_X2,
                    Adafruit_BMP280::SAMPLING_X16,
                    Adafruit_BMP280::FILTER_X16,
                    Adafruit_BMP280::STANDBY_MS_500);
}

digitalWrite(ok ? LED_GREEN : LED_RED, HIGH);
nextBtn.begin();
if (ok) Serial.println("Both sensors ready.");
}

void loop() {
    unsigned long now = millis();
    Button::Event ev = nextBtn.read();

    bool shouldRead = (now - lastRead >= READ_INTERVAL);
    if (ev == Button::SHORT_PRESS) shouldRead = true;

    if (shouldRead) {
        lastRead = now;

        // AHT20
        sensors_event_t humidity, temp;
        aht.getEvent(&humidity, &temp);

        // BMP280
        float pressure = bmp.readPressure() / 100.0F; // hPa
        float bmpTemp  = bmp.readTemperature();
        float altitude  = bmp.readAltitude(1013.25); // Sea level hPa

        Serial.printf("[%lu ms]\n", now);
        Serial.printf("  AHT20  Temp: %.1f°C  Humidity: %.1f%%\n",
                      temp.temperature, humidity.relative_humidity);
        Serial.printf("  BMP280 Temp: %.1f°C  Pressure: %.1f hPa  Alt: %.0f m\n",

```

```
        bmpTemp, pressure, altitude);  
    Serial.println();  
}  
}
```

★ Key point

Two completely different sensors, same two wires, working simultaneously without interference. This is I2C's core value. The address system ensures the ESP32 can address each device independently even though the physical bus is shared.

5 Common I2C problems and solutions

I2C is usually straightforward but when it goes wrong, the symptoms can be confusing. Here are the most common issues and how to resolve them.

Symptom	Likely cause	Solution
Scanner finds nothing	Wrong SDA/SCL pins, missing <code>Wire.begin()</code>	Verify <code>build_flags</code> have <code>I2C_SDA=21</code> <code>I2C_SCL=22</code> . Confirm <code>Wire.begin(I2C_SDA, I2C_SCL)</code> is called in <code>setup()</code> .
Device present but <code>begin()</code> returns false	Wrong I2C address in code	Run scanner to confirm actual address. Check device datasheet for address variants.
Readings are always 0 or garbage	Device not fully initialised	Add a 100ms delay after <code>Wire.begin()</code> before calling <code>begin()</code> . Some sensors need settling time.
Works alone, fails with multiple devices	Address conflict or bus overload	Run scanner to check for duplicate addresses. Consider adding external pull-ups if bus is long.
Intermittent readings	Loose wiring or marginal pull-ups	Check all connections. Reduce I2C bus speed with <code>Wire.setClock(100000)</code> for more tolerance.

5.1 The `Wire.begin()` rule

Always call `Wire.begin()` explicitly on the E4

The ESP32 Arduino core does not automatically use GPIO 21/22 for I2C. You must always call `Wire.begin(I2C_SDA, I2C_SCL)` before any I2C library `begin()` call. Without this, the core defaults to its own SDA/SCL values which may not match the E4 board.

6 Practical exercise — environmental monitor with button control

This exercise builds a complete AHT20-based environmental monitor with button-driven controls — combining T03's Button class with T04's sensor reading into a single cohesive project.

NEXT short press: force an immediate reading. NEXT long press: toggle between slow (5s) and fast (1s) polling intervals. ENT short press: reset the reading counter.

```
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_AHTX0.h>
#include "Button.h"

Adafruit_AHTX0 aht;
Button nextBtn(BTN_NEXT);
Button entBtn(BTN_ENT);

unsigned long lastRead = 0;
unsigned long readInterval = 5000; // Default 5 seconds
int readCount = 0;
bool fastMode = false;

void printReading(float tempC, float humidity) {
    readCount++;
    Serial.printf("[%d] Temp: %.1f°C Humidity: %.1f%% Mode: %s\n",
        readCount, tempC, humidity, fastMode ? "FAST(1s)" : "SLOW(5s)");
}

void setup() {
    Serial.begin(115200);
    Serial.printf("T04 Exercise - Env Monitor FW: %s\n", FW_VERSION);

    Wire.begin(I2C_SDA, I2C_SCL);

    pinMode(LED_GREEN, OUTPUT);
    pinMode(LED_RED, OUTPUT);
    pinMode(LED_BLUE, OUTPUT);
    digitalWrite(LED_GREEN, LOW);
    digitalWrite(LED_RED, LOW);
    digitalWrite(LED_BLUE, LOW);

    nextBtn.begin();
    entBtn.begin();

    if (!aht.begin(&Wire)) {
        Serial.println("ERROR: AHT20 not found");
        digitalWrite(LED_RED, HIGH);
        while (true) { delay(100); }
    }

    digitalWrite(LED_GREEN, HIGH);
    Serial.println("Ready. NEXT=read/speed ENT=reset counter");
}
```

```

}

void loop() {
    unsigned long now = millis();
    Button::Event nextEv = nextBtn.read();
    Button::Event entEv = entBtn.read();

    // NEXT short: immediate read
    bool shouldRead = (nextEv == Button::SHORT_PRESS);

    // NEXT long: toggle fast/slow mode
    if (nextEv == Button::LONG_PRESS) {
        fastMode = !fastMode;
        readInterval = fastMode ? 1000 : 5000;
        digitalWrite(LED_BLUE, fastMode ? HIGH : LOW);
        Serial.printf("Mode: %s\n", fastMode ? "FAST (1s)" : "SLOW (5s)");
    }

    // ENT short: reset counter
    if (entEv == Button::SHORT_PRESS) {
        readCount = 0;
        Serial.println("Counter reset.");
    }

    // Timed read
    if (now - lastRead >= readInterval) shouldRead = true;

    if (shouldRead) {
        lastRead = now;
        sensors_event_t humidity, temp;
        aht.getEvent(&humidity, &temp);
        printReading(temp.temperature, humidity.relative_humidity);
    }
}

```

6.1 What to observe

- GREEN LED on at startup confirms AHT20 found and initialised correctly
- Serial output shows temperature and humidity on the timed interval
- NEXT short press triggers an immediate reading outside the normal schedule
- NEXT long press switches between 1s and 5s intervals — BLUE LED indicates fast mode
- ENT short press resets the reading counter back to zero
- All button events respond immediately — nothing is blocked during sensor reads

Try this

Breathe on the AHT20 sensor — humidity should rise noticeably within a few seconds in fast mode. Cup your hand around the sensor to warm it and watch the temperature reading climb. This is a good test that your sensor is responding correctly.

7 T04 summary

Concept	Key takeaway
I2C wires	SDA (GPIO 21) and SCL (GPIO 22). Two wires shared by all devices. Always call <code>Wire.begin(I2C_SDA, I2C_SCL)</code> .
Addressing	Every device has a unique 7-bit address. No two devices on the same bus can share an address.
Pull-ups	E4 has 4.7kΩ pull-ups fitted. No external resistors needed for onboard or expansion header devices.
I2C scanner	Always run the scanner before writing sensor code. It reveals exactly what is on the bus and confirms addresses.
AHT20	Address 0x38. Use Adafruit AHTX0 library. <code>getEvent()</code> returns temperature and humidity in one call.
Multiple devices	Add devices to the same SDA/SCL bus. The address system handles arbitration. No code conflicts.
Error handling	Always check <code>begin()</code> return value. Use <code>LED_RED</code> as a visual error indicator. Halt if a required sensor is missing.

What's next

- T05 — OLED display: you can now read temperature and humidity. The natural next step is displaying it on the 0.96" SSD1306 OLED — your first complete sensor-to-screen project on the E4.
- T06 — One-wire and analog: after the OLED, the DS18B20 temperature sensor and LDR light sensor round out the Tier 1 sensor coverage.